Fair Environments Saturate: Lessons from Thirty Verifiable Agentic Environments Built to Defeat Frontier Models

Rohan Pandey
Gauntlet Vector Labs
gauntletvectorlabs@gmail.com

November 2026 (preprint draft)

Abstract

Reinforcement-learning and evaluation environments for frontier agents are increasingly built to a demanding specification: the task must be *fair* (fully specified, solvable by a reference oracle from the observations the agent sees, graded deterministically without an LLM judge) and yet *hard*, meaning a current frontier model scores well below the oracle. Over two months we built and probed 32 such environments across eleven capability axes, including estimation, stateful specification following, reverse engineering, active learning, inductive rule inference, long-horizon repair of multi-thousand-line codebases, compiler construction, and novel Lean-verified theorem proving at the hard tail of olympiad difficulty. We also red-teamed roughly 90 further designs offline before spending any model budget. Against shell-equipped agents running GPT-5.6-SOL, GPT-6.1-SOL, and Claude Opus 5/5.5, no environment that met the fairness bar held a frontier model: the frontier matched the oracle or came within a few points of it, with one exception at 0.79 of the oracle. Each apparent "hold" traced back to one of three things. Some were *artifacts*: turn caps, wall-clock throughput limits, hidden grading parameters, or rules the task never stated. Some were *exploits* in the grader. The remainder were *computational hardness*: tasks whose best-known algorithm is exponential, such as random syndrome decoding, which held both frontier models at about 0.26 of the oracle with no hidden information. We state the regularity as a falsifiable conjecture, *fair* $\Rightarrow$ *codeable* $\Rightarrow$ *saturated*. We give a partition argument for why the space of fair, compute-light tasks is the frontier's own solved region. We also catalogue the reward hacks and evaluation artifacts we encountered, together with the mitigations that closed them. We release the saturated environments as an open, deterministic suite.

1 Introduction

Benchmarks for language models saturate quickly [8]. The response has been to build harder tasks: expert-written questions [5, 14], formal mathematics [18, 19], real software repositories [7], and agentic tasks executed in containers with a shell [12]. In parallel, a market has formed for *RL environments*: self-contained, containerized tasks with a programmatic reward, sold to model developers as training and evaluation signal. Buyers in this market impose two requirements that pull in opposite directions:

1. **Fairness.** The task must be fully specified. A reference *oracle* must reach (near-)full reward using only what the agent can observe. Grading must be deterministic and must not depend on a hidden convention the agent is expected to guess.

2. **Headroom.** A current frontier agent, given a shell, tools, and a realistic time budget, should score well below the oracle. A common threshold is below 40% of the oracle.

In this paper, we report what happened when we tried, repeatedly and with substantial offline resources, to satisfy both requirements at once. The short answer is that we could not, except by relying on computational hardness. The longer answer is a map of *why*: which design choices fail, how they fail, and which apparent successes were measurement artifacts.

We build and probe 32 environments across eleven capability axes, each anchored by an oracle and a null agent (Section 3). No environment that meets the fairness bar holds the current frontier. All but one reach a hold ratio of at least 0.9 for some frontier model, and the exception scores 0.79 (Figure 1). From these results we state a falsifiable conjecture, *fair* $\Rightarrow$ *codeable* $\Rightarrow$ *saturated*, and give a partition argument that locates the only robust exception, exponential computational hardness (Section 4). We then study the one clean hold in detail: low-weight syndrome decoding of random linear codes, which holds both frontier models at about 0.26 of the oracle with nothing hidden (Section 5).

We end the introduction by remarking on two further results that we believe are useful to anyone building environments. First, most of the headroom we initially measured was false: it came from turn caps, hidden grading parameters, unstated rules, or exploitable graders. We catalogue each case together with the mitigation that closed it (Section 6). Second, the usual tools for calibrating difficulty are unreliable. Builder estimates and cheap proxy agents are miscalibrated in both directions, and headroom must be measured relative to the oracle (Section 7). Finally, we release the saturated environments as a fully explicit, deterministic, and verifiable suite.

2 Setting and Definitions

Environment format. All environments follow the task format of Harbor [6], the evaluation harness for Terminal-Bench 2.0 [12]. Each task consists of a container image, a natural-language instruction, a hidden `tests/` directory containing the verifier, and a reference `solution/`. The agent works in `/app` with a shell and runs for a fixed wall-clock budget, typically 30–60 minutes and up to 4 hours for the largest build tasks. The verifier then writes a scalar reward in $[0, 1]$. Two anchors are run for every environment: the *oracle*, which executes the reference solution, and *nop*, an agent that does nothing.

Agents. GPT models were run through the Codex CLI agent and Claude models through the Claude Code agent, both with full shell access, on a hosted Harbor-compatible execution service. We probed GPT-5.6-SOL (September 2026) and GPT-6.1-SOL (October–November 2026), together with Claude Opus 5 and Opus 5.5. Agents had network access unless stated otherwise.

Definition 1 (Fair environment)**.** An environment is *fair* if (i) every rule needed to compute the reward is disclosed in the instruction or is discoverable by interacting with the environment; (ii) a reference oracle that uses only agent-observable inputs attains reward ≥ 0.9; (iii) the reward is a deterministic function of the agent's artifact; and (iv) the grader cannot be manipulated from inside the agent's sandbox.

Definition 2 (Hold)**.** Let $\bar{r}_M$ be the mean reward of model M over n independent trials and $r^\star$ the oracle reward. The *hold ratio* is $\mathrm{H}_M = \bar{r}_M / r^\star$. An environment *holds* M if $\mathrm{H}_M < 0.4$, and *saturates* M if $\mathrm{H}_M \geq 0.9$.

Table 1: Fair environments probed on frontier agents; none held. $r^\star$ = oracle reward; scores are mean rewards (number of trials in parentheses). "Fair" marks whether the environment met the definition in Section 2 as finally built.

Axis	Environment (task)	$r^\star$	Frontier mean (n)	Fair
Estimation	robust mixture regression (EM)	1.00	GPT-5.6: 0.98	✓
	Hénon system identification	1.00	GPT-5.6: 0.998	✓
	spectral super-resolution (ESPRIT oracle)	1.00	GPT-5.6: $\approx r^\star$ (5)	✓
	random-walk + noise variance (Kalman MLE)	0.954	GPT-5.6: 0.956 (5)	✓
	Lorenz-63 partial-obs. assimilation	0.980	GPT-5.6: 0.982 (5)	✓
	coverage-biased duration estimation	0.955	GPT-5.6 0.955 (2); Opus-5.5 0.955 (2)	✓
	IMU gyro calibration under spikes	1.00	GPT-5.6: 1.00 (8)	✓
Phase transition	sparse recovery above ℓ_1 threshold (Bayes-AMP)	0.946	GPT-5.6: 0.946	✓
	real phase retrieval at spectral-init threshold	1.00	GPT-5.6: 0.996	✓
Stateful spec	billing reconciliation, 12 rules, 430 events	1.00	GPT-5.6: 1.00 (8)	✓
	interpreter for 15 non-standard semantics	1.00	GPT-6.1 1.00 (4); Opus-5.5 1.00 (4)	✓
Security	indirect prompt injection, 5 vectors	1.00	GPT-5.6: 1.00 (8 each)	✓
Reverse eng.	stripped 64-opcode VM, 16 quirks	1.00	GPT-5.6: 1.00 (8)	✓
Active learning	noisy 1-bit comparison queries	1.00	GPT-5.6: 1.00 (8)	✓
Induction	hidden list-transform, depth 4 / 5	1.00	GPT-6.1: 1.00 (3) / 1.00 (3)	✓
Long-horizon code	2,546-LOC scheduler, 7 coupled defects	1.00	GPT-6.1 1.00 (4); Opus-5.5 1.00 (4)	✓
	compiler forensics, 12 latent defects	1.00	GPT-6.1: 0.92 / 1.00 (no reports)	✓
	compiler back-end recovery, 120 samples	1.00	GPT-6.1: 0.79	✓
Construction	ML-family compiler to AArch64 (v1–v3)	1.00	GPT-6.1: 1.00 / 0.93 / 1.00	✓
	+ type classes, GADTs, effects (4 h budget)	1.00	GPT-6.1 0.887 (3); Opus-5.5 0.931 (5)	✓
Formal proof	10 novel Lean theorems (bespoke recurrences)	1.00	GPT-6.1 1.00 (4); Opus-5.5 0.975 (4)	✓
	17 hard novel Lean theorems (descent, Vieta, Pell)	1.00	GPT-6.1 1.00 (4); Opus-5.5 1.00 (4)	✓

We use the ratio rather than the absolute reward deliberately. Tightening a tolerance lowers the frontier's absolute score, but it usually lowers the oracle's score by the same factor and leaves the ratio unchanged (Section 7).

3 Results

In this section, we describe the results of every frontier probe. Tables 1 and 2 summarize all environments probed on frontier agents. We group them by the capability axis each was designed to stress. Sample sizes are small: probes cost real money and most results were unambiguous. Where trial-level variance mattered we report the range. Figure 1 plots the same data as hold ratios.

3.1 What Saturated, and How

Reading the agent traces shows that saturation was never a matter of luck. In each axis the frontier agent did what a strong human expert with unlimited patience would do:

- **Estimation.** The agent derived the correct likelihood from the disclosed generative model and



Figure 1: Hold ratio for every environment probed on a frontier agent. Fair environments (top) cluster at the oracle. Artifact rows show the score with the artifact present (hollow) and after it was removed (filled); every one moved into or near the saturated band. Only the three computationally hard environments (bottom) stay in the hold region. The C++ exploit environment is omitted because its exploit reached 1.0 rather than lowering the score.

Table 2: Environments that held a frontier model, by cause: artifact (A), exploit (E), or computational hardness (C). Only type C survives the fairness definition.

Cause	Environment (task)	$r^\star$	Frontier mean (n)	Fair
Artifact (A)	many-time pad, 24-turn cap	1.00	Opus-5.5 0.27 capped → GPT-5.6 1.00 uncapped	–
	one-sided noise, rule unstated	0.999	GPT-5.6: 0.32–0.46 (5–10)	×
	odometer calibration, grading cadence hidden	0.963	GPT-6.1 0.53 (5); Opus-5.5 0.62 (5)	×
	same, cadence disclosed	1.00	GPT-5.6: 1.00	✓
	compiler, 60-min budget (v4–v8)	1.00	GPT-6.1: 0.42–0.82	(×)
Exploit (E)	C++ device sim, private state	1.00	template-instantiation access → 1.00	×
Comp. hard (C)	MAX-XORSAT / LPN, n=512	1.00	GPT-5.6: 0.36–0.47 (15)	✓
	reduced-round ARX cipher key recovery, R=6	1.00	GPT-5.6 0 (3); Opus-5 0 (1), timeouts	✓
	syndrome decoding, n=2048, $w^\star$=102	1.00	GPT-6.1 0.26 (2+); Opus-5.5 0.25 (2+)	✓

implemented the estimator. On the Hénon task it reasoned explicitly about errors-in-variables bias and switched to multiple shooting. On the coverage-bias task it derived inverse-probability weighting from an operational description that did not state the formula.

- **Research-level methods.** The agent either recalled the optimal method (Bayes-AMP with the Onsager correction [3, 4]) or searched the literature from inside the container for it (optimal spectral preprocessing for phase retrieval [10, 13]). Making the optimal method obscure is not a moat when the agent can read arXiv.
- **State tracking and specifications.** The agent did not track state "in its head". It wrote a deterministic simulator of the disclosed grammar and ran it. This sidesteps the long-context state-tracking failures reported for models without tools.
- **Induction.** On the hidden list-transform tasks, GPT-6.1-SOL identified most rules by inspection, and Opus-5.5 ran a program-synthesis search over about 250 primitive operations. Either way the stationary hidden rule was recovered exactly.
- **Formal proof.** On 17 novel theorems that required non-standard insight (infinite descent on cubes, Vieta jumping, unreachability of Pell forms, boundary extrema of cyclic inequalities), both models produced complete Lean proofs that type-checked under a locked statement and an axiom whitelist, in every trial.
- **Scale.** On a 2,546-line scheduler with seven coupled defects spread across five files, and on compiler-construction tasks with hand-written assembly runtimes, the agent used its full budget and stayed correct. Growing the scope of a fair, specified build task did not produce a hold once the time budget was adequate (Section 6).

3.2 Offline Red-Teaming

Before spending probe budget we routinely attacked candidate designs with ensembles of frontier-class agents, each instructed to break the task by a different route. Across four sweeps covering about 90 candidates (games, cryptographic puzzles, query complexity, online control, interactive abduction, arithmetic-circuit minimization, morphophonology, geometric construction, and others), no candidate survived as a fair hold. Recurring causes of death were the same as in Tables 1 and 2:

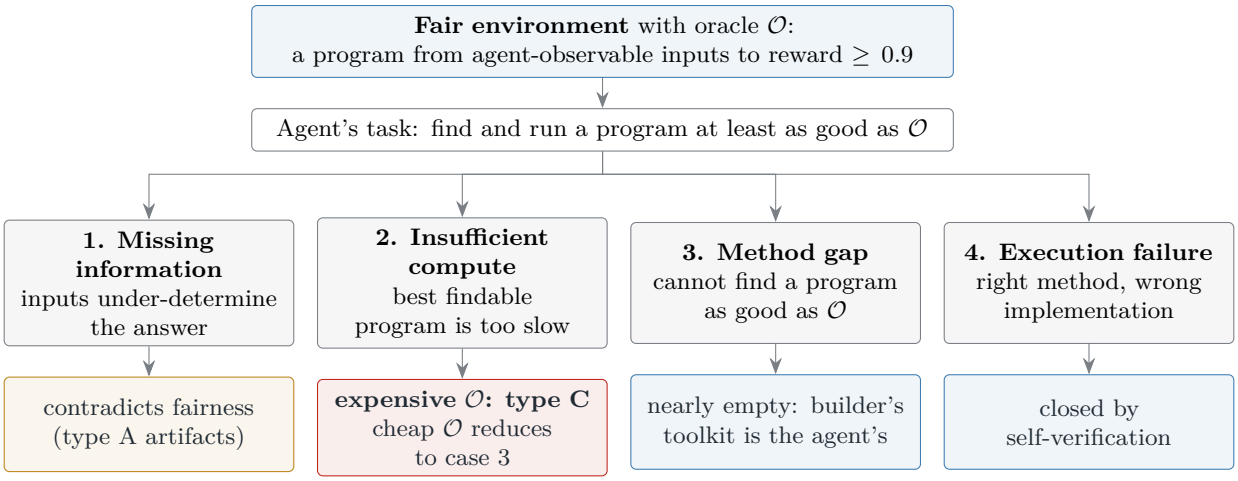


Figure 2: The partition argument. A fair environment reduces the agent's task to matching a known program. Of the four ways to fail, one contradicts fairness, two are closed in practice by literature access and self-verification, and only exponential compute (red) survives as a fair hold.

a few-line codeable solver, recall of the known optimal method, or a dial between "trivial" and "intractable" with no fair middle setting.

Two results from these sweeps are worth recording.

Interactive abduction. In seven interactive worlds with hidden mechanics, a passive logger had provably zero mutual information about some mechanics: they never fired under random exploration. Even so, a generic count-based novelty explorer of a few lines matched the oracle in six of the seven. The exception, a typed-graph cellular automaton with a passive Bayes-optimal ceiling of 0.109, stayed hard only under a fragile train-then-test protocol.

Arithmetic-circuit minimization. When the target polynomial was an obfuscated elementary symmetric polynomial, an iterated matrix multiplication, or a determinant, every one of eight attack strategies, including plain common-subexpression elimination with greedy search, found circuits *smaller* than our own oracle construction.

4 Why Fair Environments Saturate

Conjecture 1 (Fair $\Rightarrow$ codeable $\Rightarrow$ saturated)**.** *For a shell-equipped frontier agent with a realistic time budget, any environment that is fair (Definition 1) and whose oracle runs in time polynomial in the instance size with modest constants will be saturated,* $\mathrm{H} \geq 0.9$*, by the current frontier.*

In this section, we explain why we believe this regularity is structural. The informal argument is a partition. Fix a fair environment and its oracle $\mathcal{O}$. Because the environment is fair, $\mathcal{O}$ is a program that maps agent-observable inputs to a high-reward output. The agent's task therefore reduces to *finding and running a program at least as good as* $\mathcal{O}$. The agent can fail to do so for only four reasons, shown in Figure 2.

1. **Missing information.** The inputs under-determine the answer. This contradicts clause (i) of fairness. All of our type-A artifacts fall here: an unstated noise model or a hidden grading cadence.

2. **Insufficient compute.** The best program the agent can find needs more time than the budget allows. If $\mathcal{O}$ is itself cheap, this case requires $\mathcal{O}$ to use a method the agent cannot find, which is case 3. If $\mathcal{O}$ is expensive, the environment is computationally hard (type C).
3. **Method gap.** The agent cannot find a program as good as $\mathcal{O}$. Our results suggest this case is now nearly empty for anything the *builder* can implement: the builder's toolkit (textbooks, papers, solvers, proof assistants) is also the agent's, and the agent can search the literature. We call this *construction-difficulty symmetry*.
4. **Execution failure.** The agent knows the method but implements it wrongly and does not notice. Fairness gives the agent everything it needs to build its own verifier (worked examples, a disclosed grammar, a type checker), so self-verification closes most of this gap.

A related observation came from precision-gap estimation tasks.

Observation 1 (Matters ⇔ estimable)**.** *In a fair estimation task, a nuisance parameter that materially affects the reward is either estimable from the data, so a competent agent estimates it, or it is not, in which case ignoring it costs little. Designs that need both ("it matters,* and *the agent cannot estimate it") violate fairness.*

We hit this directly when trying to harden an odometer-calibration task. Making a drift term large enough to matter also made it estimable from a single run. Making it unestimable required a random-walk confound so large that a solver which ignored drift entirely scored 0.96.

Escapes. The argument leaves three regions in which a fair environment can still hold the frontier: (a) *computational hardness*, where the best known method is exponential and the oracle holds a planted witness; (b) *open-ended judgment*, which conflicts with deterministic grading; and (c) *messy real-world scale* (very large real repositories, flaky dependencies, genuinely ambiguous issues), which conflicts with clean, procedural generation and invites contamination. Real-repository benchmarks were built around exactly this kind of mess, and frontier pass rates on them started far below saturation [7], which is consistent with (c).

5 Case Study: Random Syndrome Decoding

The one clean, fully disclosed hold we found uses (a). The agent receives ten instances of a random rate-1/2 binary parity-check matrix $H \in \mathbb{F}_2^{1024\times 2048}$ and a syndrome $s = He^\star$, where $e^\star$ is a planted error of weight $w^\star = 102$. Since $w^\star$ lies far below the Gilbert–Varshamov distance (about $0.11n$), $e^\star$ is with overwhelming probability the unique minimum-weight coset element. The agent must return any e with $He = s$ of the lowest weight it can find. The per-instance reward is

$$r = \mathrm{clip}\left(\frac{w_{\mathrm{base}} - \mathrm{wt}(e)}{w_{\mathrm{base}} - w^\star},\, 0,\, 1\right), \qquad w_{\mathrm{base}} = n/4,$$

so a free particular solution from Gaussian elimination scores about 0 and the planted error scores 1 (Figure 3). Everything, including H, s, $w^\star$, the reward, and the grading compute budget, is disclosed. The oracle simply returns $e^\star$, which it holds by construction. This is the asymmetry that computational hardness allows and that fair compute-light tasks lack.

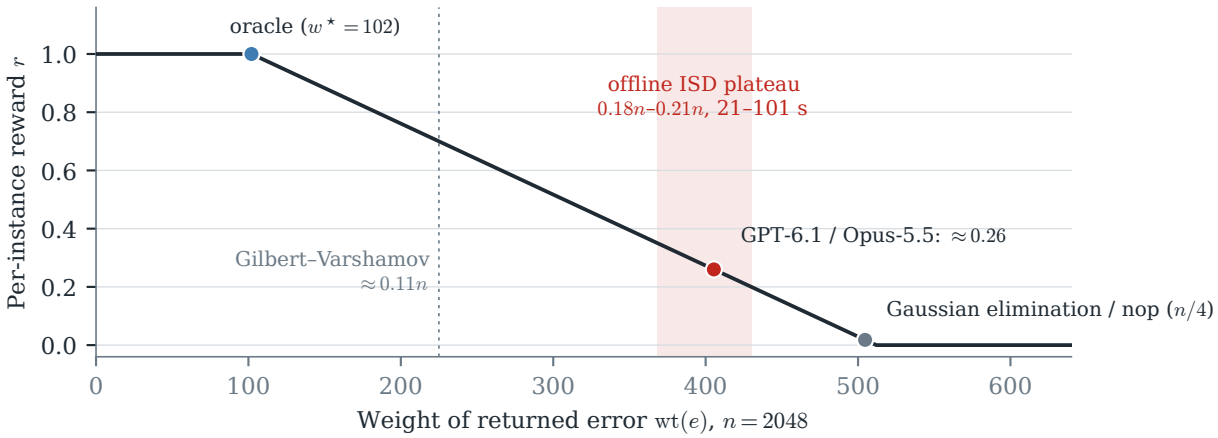


Figure 3: Reward as a function of the weight of the returned error vector. Both frontier models land at about 0.26, inside the plateau that our own offline information-set decoder reaches and well above the Gilbert–Varshamov distance. The planted error sits far below either.

Why it holds. Information-set decoding (ISD) [1, 11, 15, 17] is the best known approach and is exponential in n. In offline tests our own ISD implementation (Prange plus Lee–Brickell plus basic Stern) plateaued at weight $0.18n$–$0.21n$ for $n \in \{1024, 1536, 2048\}$. The curves were flat between 21 s and 101 s at n=2048, and the floor sat at about twice the GV distance and four times $w^\star$. Recalling or looking up the optimal method does not help, because the optimal method is itself exponential. This reverses the failure mode that killed our phase-transition environments, where finding the optimal method was the whole difficulty.

Measurements. With public network access, GPT-6.1-SOL scored 0.267 (trials 0.263 and 0.271) and Opus-5.5 scored 0.245. A hardened revision changed three things: the solver runs in a sandbox as an unprivileged user with the test directory unreadable, seeds are 62-bit, and the shipped default solver is trivial (nop = 0.018). On that revision both models scored about 0.26 against an oracle of 1.0. The traces show both agents implementing or tuning ISD variants; neither got past the exponential floor.

Caveats. Three limitations are worth stating. First, the reward has a narrow effective dynamic range: most of the achievable signal sits in a band of about 0.02 near the ISD floor. Second, how hard the task is depends on the compute budget, although only logarithmically. Third, the result mostly measures the absence of algorithmic breakthroughs in decoding, which is a fact about mathematics rather than about the model. Buyers who want "reasoning, not compute" may reasonably reject it. That rejection is our point: under the conjecture, fair-and-hard forces the hardness to be computational.

The two other type-C environments in Table 2 agree with this. A planted MAX-XORSAT (learning parity with noise) instance with n=512 capped GPT-5.6-SOL at the value reachable by Gaussian elimination plus local refinement (0.36–0.47 over 15 trials, with no crack and no abandonment). The same family at n=160 was decoded outright. A reduced-round ARX cipher with four known plaintext pairs was solved in 32 s by CryptoMiniSat at five rounds and resisted it for more than 30 minutes at six. That jump produced binary outcomes: solved at R=5, timed out at R=6.

6 Artifacts and Exploits That Create False Headroom

Most of our early "successes" were spurious. We list them because each is easy to fall into, and some appear in published environments.

6.1 Evaluation Artifacts

Turn caps. A many-time-pad recovery task scored 27% on Opus-5.5 under a 24-turn cap (n=64). Without the cap, GPT-5.6-SOL solved 8/8 in about 13 steps of ordinary crib-dragging. *Mitigation:* measure with unlimited turns and full tools.

Unstated rules. A line-fitting task with one-sided (non-negative) noise held GPT-5.6-SOL at 0.32–0.46 and passed an automated difficulty check. It was hard only because the instruction did not say the noise was one-sided, so ordinary least squares was a reasonable reading. When the rule was stated, the score was 0.9999. Scores were bimodal: an agent either guessed the rule or it did not.

Hidden grading parameters. An odometer-calibration task held GPT-6.1-SOL at 0.53 only because the correct correction depended on an undisclosed grading step size. The oracle itself scored between 0.48 and 0.96 depending on that step, and low-scoring agent traces explicitly noted that they were guessing it. With full disclosure the frontier scored 1.0.

Phantom parameters. Advertising a drift parameter that the simulator never applied lured agents into fitting it, which drove most trials to 0. This is a decoy, not difficulty.

Wall-clock throughput. A compiler-construction task scored 0.42–0.82 under a 60-minute budget with all-or-nothing milestones. Under a 4-hour budget with partial credit it scored 0.89 (GPT-6.1-SOL) and 0.93 (Opus-5.5). The difficulty was mostly the clock.

Tolerance wider than the hard feature. In a spectral task the frequency tolerance was 1.28 FFT bins, wider than the spacing of the close tone pairs. Reporting the merged peak twice earned most of the reward. *Mitigation:* test the reward against deliberately degenerate answers.

Timeouts and abandonment. Binary exact-match tasks near the edge of feasibility produced agent timeouts rather than wrong answers. Graded tasks with an obvious trap produced *abandoned* trials in which the agent explored for about 12 minutes and never committed an answer. Neither outcome gives a usable score. *Mitigation:* use continuous rewards and a pre-seeded, always-gradeable deliverable.

Safety classifiers. Security-flavored wording ("reverse engineer", "key", "CTF") triggered refusals or provider-side safety errors that score 0 without measuring capability. Neutral wording of the identical task removed them. *Mitigation:* check stop reasons whenever a strong model scores near 0 at near-zero cost.

6.2 Reward Hacks Found in Our Own Graders

Each of the following was found by red-teaming or review before release. All but one were fixed inside the task; reward-file tampering can only be closed by the evaluation platform.

Generator import. A solver executed by the verifier could import the hidden instance generator from `tests/` and read off the planted witness. *Fix:* run the agent's program as an unprivileged user with the test directory unreadable.

Low-entropy seeds. Instance seeds drawn from a small space could be brute-forced by regenerating candidates and matching them against the public instance. *Fix:* 62-bit seeds derived with a cryptographic hash. Python's `hash()` is randomized per process and must never be used for seeding.

Reward-file tampering. When the verifier shares a container with a root-privileged agent, the agent can plant a Python `.pth` file or otherwise rewrite the reward path. Sandboxing the agent's program does not help, because the hook runs inside the verifier's own interpreter. On our syndrome-decoding environment a planted hook raised the reward from 0.018 (do nothing) to 1.0. *Fix:* grade in a separate verifier container that receives only the agent's artifacts. This needs platform support and cannot be done from inside a single-container task.

C++ access control. Explicit template instantiation can legally take pointers to private members, which defeats both a regex ban-list and C++ access control and gives the agent the simulator's hidden truth. *Fix:* keep device state inside the grader process and expose only an opaque step/read interface.

Module shadowing. A verifier that ran `python -m` with working directory `/app` imported an agent-written module instead of the reference implementation. *Fix:* run in isolated mode (`python -I`), precompute reference outputs, and fail closed.

Proof weakening. In the Lean environments an agent could restate a weaker theorem or rely on `sorry`, extra axioms, or `native_decide`. *Fix:* add a type-level lock `theorem _ : <canonical statement> := @name` and an axiom whitelist (`propext`, `Classical.choice`, `Quot.sound`).

Non-finite outputs. A NaN or infinity in a numeric answer could slip through comparison-based scoring. *Fix:* reject non-finite values outright.

No frontier agent we inspected attempted any of these hacks during a probe. Every trace we read was a legitimate solution attempt. But an RL policy optimized against the grader would be expected to find them [9, 16].

7 Lessons for Calibrating Difficulty

Builders underestimate the frontier. Before each probe we recorded the builder's estimate of the frontier score. On the six most carefully built environments the estimates ranged from 0.13 to 0.45, and the measured scores were 0.975 to 1.0. This happened every time. Self-estimates of difficulty carry essentially no information.

Proxy agents are miscalibrated in both directions. We used a cheaper local agent as a screening gate. On a calibration environment with a known GPT-5.6-SOL score of 0.44, the proxy scored 0.47–0.83. It was a stronger coder than the target on that task, so it produced false *negatives* (it declared feasible tasks easy). On Lean tasks the proxy stalled while setting up the toolchain, which produced false *positives*. Proxies are useful for discarding candidates that are clearly easy. A borderline result needs a real probe on the target model.

Measure headroom relative to the oracle. Halving an error floor in a log-scaled reward lowered the frontier's absolute score from 0.84 to 0.77 and lowered the oracle's score in the same proportion. The hold ratio stayed at 0.84.

Fixed instances collapse; distributions calibrate. A capable model solves a fixed puzzle at close to 100% or close to 0%, because its error rate on any technique it knows is low. When we needed intermediate pass rates, the tunable object was an *instance distribution* with a difficulty parameter, not a single harder puzzle.

8 Related Work

Benchmark saturation and dynamic evaluation. Dynabench [8] argued that static benchmarks saturate and proposed adversarial human-in-the-loop collection. Expert-written benchmarks such as Humanity's Last Exam [14] and FrontierMath [5] rely on difficulty that the authors can certify. Our results suggest that certifiable, fair difficulty is exactly what a tool-using agent can match.

Formal mathematics. miniF2F [19] and PutnamBench [18] grade proofs with a proof assistant. Our Lean environments use the same grading but with newly authored statements, to rule out contamination. They were nonetheless saturated by the current frontier.

Abstraction and interaction. ARC [2] targets skill acquisition from few examples. In our interactive abduction sweep, a shell agent's ability to log transitions and synthesize programs removed most of the gap between active and passive exploration.

Reward hacking. Specification gaming [9] and reward hacking [16] are well documented in RL. Section 6 adds a concrete, environment-engineering-level catalogue for containerized agent tasks.

Code-based cryptography. The hardness of decoding random linear codes underlies code-based cryptography. The best attacks are ISD variants [1, 11, 15, 17], which is why syndrome decoding gives a principled, tunable hardness dial.

9 Limitations

This is a single-team study. Our environments are only as good as our building of them, and a stronger builder might find fair holds we missed. Sample sizes per environment are small (typically 2–8 trials), which is enough to separate 0.25 from 1.0 but not to resolve fine differences. All probes ran through a single hosted execution service and two agent harnesses. Model versions change quickly: several holds against GPT-5.6-SOL disappeared one generation later. The conjecture is stated for shell-equipped agents. Models without tools behave differently, and many published long-context failures do not carry over to agents that can write code. Finally, the offline red-team sweeps used frontier-class agents as attackers. Their verdicts are evidence, not proof.

10 Conclusion and Future Directions

Fairness, as environment buyers define it, requires that the task be solvable from what the agent sees by a method the builder can implement. In our experiments that same property let a tool-using frontier agent find and run an equally good method. Headroom that survives careful review came from computational hardness and from nothing else. For people building environments, the practical advice is to stop searching for clever fair tasks in the compute-light region and to choose one of

three options. The first is to accept principled computational hardness with planted witnesses. The second is to invest in messy, real-world-scale tasks. The third is to evaluate *adaptively*, generating new instances in response to the model under test rather than shipping a static task. We release the saturated environments, which are still deterministic and diverse training content with continuous rewards, so that others can test the conjecture.

In future work, we plan to test the conjecture on real-repository tasks at a scale that a single agent cannot read end to end, and to build adaptive evaluations that generate instances in response to the model under test. Additional work is also necessary to measure how quickly the one clean hold, syndrome decoding, erodes as agents gain compute and better decoders.

Release. Environment sources, generators, oracles, and verifiers will be available at https://rohan-pandey1729.github.io/gauntlet-site/publications.html.

References

[1] Anja Becker, Antoine Joux, Alexander May, and Alexander Meurer. Decoding random binary linear codes in $2^{n/20}$: How $1 + 1 = 0$ improves information set decoding. In *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 520–536. Springer, 2012.

[2] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[3] David L. Donoho, Arian Maleki, and Andrea Montanari. Message-passing algorithms for compressed sensing. *Proceedings of the National Academy of Sciences*, 106(45):18914–18919, 2009.

[4] David L. Donoho, Arian Maleki, and Andrea Montanari. Message passing algorithms for compressed sensing: I. motivation and construction. In *IEEE Information Theory Workshop*, pages 1–5, 2010.

[5] Elliot Glazer, Ege Erdil, Tamay Besiroglu, et al. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI. *arXiv preprint arXiv:2411.04872*, 2024.

[6] Harbor Framework Team. Harbor: A framework for evaluating and optimizing agents and models in container environments. https://github.com/laude-institute/harbor, 2026.

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

[8] Douwe Kiela, Max Bartolo, Yixin Nie, Divyansh Kaushik, Atticus Geiger, Zhengxuan Wu, Bertie Vidgen, Grusha Prasad, Amanpreet Singh, Pratik Ringshia, et al. Dynabench: Rethinking benchmarking in NLP. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4110–4124, 2021.

[9] Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. Specification gaming: the flip side of AI ingenuity. DeepMind Blog, https://deepmind.google/discover/blog/specification-gaming-the-flip-side-of-ai-ingenuity/, 2020.

[10] Wangyu Luo, Wael Alghamdi, and Yue M. Lu. Optimal spectral initialization for signal recovery with applications to phase retrieval. *IEEE Transactions on Signal Processing*, 67(9):2347–2356, 2019.

[11] Alexander May, Alexander Meurer, and Enrico Thomae. Decoding random linear codes in $\tilde{O}(2^{0.054n})$. In *Advances in Cryptology – ASIACRYPT 2011*, volume 7073 of *Lecture Notes in Computer Science*, pages 107–124. Springer, 2011.

[12] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *The Fourteenth International Conference on Learning Representations*, 2026. arXiv:2601.11868.

[13] Marco Mondelli and Andrea Montanari. Fundamental limits of weak recovery with applications to phase retrieval. *Foundations of Computational Mathematics*, 19(3):703–773, 2019.

[14] Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, et al. Humanity's last exam. *arXiv preprint arXiv:2501.14249*, 2025.

[15] Eugene Prange. The use of information sets in decoding cyclic codes. *IRE Transactions on Information Theory*, 8(5):5–9, 1962.

[16] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. In *Advances in Neural Information Processing Systems 35*, 2022.

[17] Jacques Stern. A method for finding codewords of small weight. In *Coding Theory and Applications*, volume 388 of *Lecture Notes in Computer Science*, pages 106–113, 1989.

[18] George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amitayush Thakur, and Swarat Chaudhuri. PutnamBench: Evaluating neural theorem-provers on the Putnam mathematical competition. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024.

[19] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. miniF2F: a cross-system benchmark for formal olympiad-level mathematics. In *International Conference on Learning Representations*, 2022.